Formula 1 Racing Car Detection and Tracking

SDAIA Academy | Computer Vision for Developers | Final Project Report

Trainee: Abdulrahman bin Turki Al-Mutairi
Proposed model: YOLOv8 object detector | Development environment: Google Colab

Report scope: This document explains the project design and evaluation plan. Experimental results must be added after running the model; they are not available in the supplied report.

1. Project Idea

This project aims to detect Formula 1 racing cars in video footage and follow their movement across consecutive frames. It belongs to the sports analytics domain and uses computer vision to make vehicle observation faster and more consistent than manual review.

A YOLOv8 object detection model is proposed to locate cars in each frame. A multi-object tracking method can then assign an ID to each detected car and attempt to keep that ID as the car moves through the video.

Application domain: Motorsport video analysis. Main tasks: Object detection and multi-object tracking. Transfer learning: Pretrained weights may be used; fine-tuning is optional and should only be reported if actually performed.

2. Problem Definition

Problem: Formula 1 cars move quickly, appear at different sizes, and may partly hide one another during overtaking or cornering. These conditions make it difficult to locate and follow every car consistently across a video.

Purpose: Build a system that automatically identifies visible racing cars and tracks their positions from frame to frame.

Expected output: An annotated video in which each detected car has a bounding box, a detection confidence score, and—when tracking is enabled—a track ID. The system may also produce a count of detected cars per frame. A track ID does not by itself establish the driver’s identity.

3. Computer Vision Implementation

Computer vision task: Object detection, one of the tasks required by the course. Tracking is an additional capability built on top of detection.

Model: YOLOv8, a suitable real-time object detection architecture. A pretrained YOLOv8 checkpoint can be loaded using the Ultralytics Python package in Google Colab. The detector predicts bounding boxes, object classes, and confidence scores for each frame.

Processing pipeline: Read video → extract frames → resize/prepare input → run YOLOv8 inference → filter car detections → associate detections across frames using a tracker → draw bounding boxes and IDs → export annotated video.

Important class limitation: A general-purpose COCO-pretrained model can recognize the broad “car” category, but it does not automatically distinguish Formula 1 cars from other car types or identify individual drivers. Reliable F1-specific classification requires suitable labeled data and testing.

4. Data and Model

Data source: Formula 1 race video footage is described in the original report, but no specific dataset name, link, video count, frame count, or license was provided. Record the exact video source(s), permission to use them, and the amount of footage analyzed before submission.

Annotation: For quantitative evaluation, prepare a labeled test set with bounding boxes around visible F1 cars. Use a consistent annotation format such as YOLO bounding-box labels. If training or fine-tuning is performed, separate training, validation, and test data to avoid evaluating on training images.

Preprocessing: Resize frames to the model input size (for example, 640 × 640, if used) and apply the normalization handled by the chosen model pipeline. Record the actual input size and frame sampling method.

Augmentation: Brightness changes, scaling, and motion blur are possible training augmentations. These were mentioned in the original report but were not documented with settings or evidence, so they should not be claimed as completed unless implemented.

Model configuration to record: YOLOv8 variant, checkpoint/weights, confidence threshold, image size, tracking algorithm, and whether the model was pretrained only or fine-tuned.

5. Evaluation

Evaluation should use a labeled test set that was not used for training. Report the actual values after testing rather than describing performance as “high” without measurements.

Measure	What to report
Precision	Of the predicted car detections, how many are correct?
Recall	Of the labeled cars, how many were detected?
mAP@0.5	Detection accuracy across confidence thresholds at IoU 0.5.
FPS / processing time	How quickly the system processes the test video.
Tracking consistency (optional)	ID switches or another tracking metric if ground-truth tracks exist.

Successful prediction case — evidence required: Include one actual frame where a racing car is correctly detected and, if tracking is used, its ID is maintained across nearby frames. Add a screenshot and briefly explain what worked.

Failure case — evidence required: Include one actual frame where a car is missed, falsely detected, or assigned an incorrect tracking ID. Occlusion during overtaking is a plausible failure scenario, but it must be demonstrated using your own output rather than presented as an observed result without

proof.

Results to insert: Precision: [not measured] | Recall: [not measured] | mAP@0.5: [not measured] | FPS: [not measured].

6. Possible Improvements

• Collect more labeled Formula 1 footage covering different camera angles, lighting conditions, weather, and car sizes.

• Fine-tune YOLOv8 on F1-specific bounding-box annotations if the general-purpose car detector misses racing cars.

• Test a suitable tracker, such as ByteTrack or BoT-SORT, to reduce incorrect ID changes when cars overlap.

• Compare model sizes and inference settings to balance detection accuracy with processing speed.

7. Submission Evidence Checklist

• Dataset source and number of labeled images or video frames.

• Actual model settings, notebook or code, and processing steps.

• Numerical evaluation results calculated on labeled test data.

• One successful prediction image and one failure image, with explanations.

• A sample annotated output video, if requested by the instructor.

8. Conclusion

This project proposes a practical use of object detection and tracking for Formula 1 race footage. YOLOv8 can be used to detect cars in individual video frames, while a tracking algorithm can help follow them over time. The project will meet the evaluation requirement once measured results and real examples of both successful and unsuccessful predictions are added. Its effectiveness should be judged from those results, not assumed in advance.

9. GitHub Reference

GitHub link provided in the original report: https://github.com/SDAIAAcademy

satam aldossari
https://github.com/
sattam555555.

Note: This is an organization page, not a verified project repository.